

UCS 503- Software Engineering

CPRecall

A Personalized LeetCode Learning, Mastery, and Spaced-Recall System – Progress
Report

Submitted by:

1024030665 Subham Kumar

1024030666 Tanush Grover

1024030669 Shubhi Singh

Group: Nexus 3 BE Third Year, CSE

Submitted to:

Raghav B. Venkataramaiyer

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	5
1.1	Background and Context	5
1.1.1	Problem Statement	5
1.2	Project Objectives	5
2	Methodology and Design	7
2.1	System Architecture	7
2.1.1	High-Level Design	7
2.1.2	Technical Stack	8
2.2	Design Completed So Far	8
2.2.1	Repository and Module Structure	8
2.2.2	Data Model	8
2.2.3	Problem Mastery Formula	9
2.2.4	Problem Mastery vs. Pattern Mastery	9
2.2.5	FSRS Integration	9
2.2.6	Frontend Views Specified	10
2.2.7	Extension Widget Specification	10
2.2.8	Authentication and Data Isolation	10
3	Results and Evaluation	11
3.1	Current Status	11
3.2	Design Verification	11
3.3	Pending Work	11
4	Conclusion	13
4.1	Summary of Work	13
4.2	Key Findings	13
4.3	Future Work and Recommendations	13
4.4	Final Remarks	14
	Bibliography	15

Introduction

1.1 Background and Context

Competitive programmers and interview aspirants typically solve a large number of LeetCode problems, but plain problem-solving volume does not by itself guarantee long-term retention of the underlying Data Structures and Algorithms (DSA) patterns. A user may solve a problem once, yet forget the pattern or the optimal approach weeks later because there is no structured mechanism that (a) evaluates how well the problem was actually solved, and (b) schedules a future review at the right time.

CPRcal (previously named CodeRecall) is being built to close this gap. It is a personalized LeetCode learning, mastery, and recall system that observes a user's solving behaviour on LeetCode, analyses the submitted code for its actual and optimal complexity, computes a quantitative mastery score for the problem, aggregates that mastery at the DSA-pattern level, and finally schedules future revisions using the FSRS (Free Spaced Repetition Scheduler) algorithm [YR23].

1.1.1 Problem Statement

Given that a user solves problems on LeetCode without any external tracking, there is currently no automated way to:

- objectively score how efficiently a specific problem was solved (in terms of time complexity, number of attempts, time taken, and hints used),
- separate a user's mastery of an individual problem from their mastery of the broader DSA pattern that problem belongs to, and
- decide *when* that user should be made to revisit the problem or pattern so that retention is maximised.

CPRcal addresses this by combining a browser extension for observation, an LLM-based code analysis step, a backend scoring and scheduling engine, and a dashboard for review.

1.2 Project Objectives

1. **Objective 1:** Capture a user's LeetCode solving activity (problem metadata, session time, submission history, and the final accepted code) automatically

through a Tampermonkey browser extension, with no manual data entry beyond an optional hints-used count.

2. **Objective 2:** Analyse each accepted solution's actual and optimal time/space complexity using an LLM, and convert this, along with the other raw solving signals, into a weighted *Problem Mastery* score.
3. **Objective 3:** Aggregate problem-level mastery into a *Pattern Mastery* score per DSA pattern, and drive a spaced-repetition review schedule for each user using the FSRS algorithm, surfaced through a React dashboard.

These objectives were defined and finalised during the design phase reported in this document; implementation proceeds directly against this specification.

Methodology and Design

2.1 System Architecture

The overall data flow that has been finalised for CPRecal is:

LeetCode → Tampermonkey Extension → LLM Code Analysis → Extension UI → CPRecal Backend

2.1.1 High-Level Design

The end-to-end solving flow that has been designed and agreed upon is as follows:

1. The user opens a LeetCode problem; the extension identifies the problem ID, title, titleSlug, difficulty, and tags/patterns.
2. The extension tracks the solving session and the total time taken, and monitors submission status/history as the user attempts the problem.
3. On an Accepted submission, the extension captures the accepted code, language, runtime, memory, submission ID, and timestamp.
4. The accepted code and question name are sent to an LLM, which returns the user's actual time/space complexity, the optimal time/space complexity, and an explanation.
5. The extension computes a *Solution Efficiency* score from the actual-versus-optimal complexity and displays it, along with the rest of the solving data, in a floating widget. The user may additionally record hints used.
6. The extension packages the raw solving data and the LLM analysis and submits it to the authenticated CPRecal backend.
7. The backend computes the final Problem Mastery score, persists it along with the supporting raw data, converts it into an FSRS rating, and lets FSRS determine the next review date, storing the resulting FSRS state.
8. The React frontend fetches the personalized dashboard, problem library, pattern dashboard, and problem-detail data from the backend.

A clear responsibility split between the three layers has also been finalised: the Tampermonkey extension is a *collector and display layer* only; the LLM is responsible purely for analysing code complexity and does not decide mastery; and the backend remains the single source of truth for authentication, persisted mastery, pattern aggregation, and FSRS review state.

2.1.2 Technical Stack

- **Framework/Language:** React for the frontend [Met24], Node.js with Express for the backend API [Ope24], and a Tampermonkey user script for the browser extension layer [Bin24].
- **Database:** MongoDB, accessed through Mongoose object models [Mon24; Aut24].
- **Scheduling Algorithm:** FSRS (Free Spaced Repetition Scheduler) for computing review intervals from mastery ratings [YR23].
- **External Integration:** An LLM API for accepted-code complexity analysis, invoked either directly from the extension or, if security/reliability considerations require it, relayed through the backend.

2.2 Design Completed So Far

2.2.1 Repository and Module Structure

A monorepo layout has been finalised to keep the React client, Express server, shared contracts, and Tampermonkey extension separated while remaining easy to scale: `client/` (pages, components, services, hooks), `server/` (models, controllers, routes, middleware, and a `services/` layer further split into `mastery/`, `fsrs/`, `llm/`, and `ingestion/` sub-modules), `extension/` (collectors, GraphQL queries, analysis, widget UI, and storage), `shared/` (constants, schemas, types), plus top-level `scripts/`, `tests/`, and `docs/` directories. Folder-level responsibilities – for example, that `server/models` holds only Mongoose persistence definitions while `server/controllers` handles only HTTP-level request/response logic – have been written down so that implementation work can proceed consistently across the team.

2.2.2 Data Model

The database design distinguishes shared, global data from user-specific data. A single shared `Problem` collection stores `problemId`, `title`, `titleSlug`, `difficulty`, and `patterns/tags`. All user-specific activity is represented through separate collections – `SolvingSession`, `Submission`, `AcceptedSolution`, `ProblemMastery`, `PatternMastery`, `FSRSState`, and `ReviewHistory` – each referencing a `userId`. Raw observations (sessions, submissions, accepted code) are retained rather than discarded, so that mastery can be recalculated later if the scoring formula changes.

2.2.3 Problem Mastery Formula

Problem mastery answers “how well did this specific user perform on this specific problem?” and has been assigned the following weights:

- Solution Efficiency – 25%
- Time Complexity – 25%
- Number of Submissions – 20%
- Time Taken (benchmarked against problem difficulty) – 15%
- Hints Used – 15%

The exact normalisation inside each component is left open for tuning during implementation, but the relative weighting has been fixed.

2.2.4 Problem Mastery vs. Pattern Mastery

A clear conceptual separation has been made between problem mastery (one user’s score for one problem) and pattern mastery (that user’s cumulative score for a DSA pattern, computed from their performance across all problems tagged with that pattern). Because a problem can carry multiple pattern tags, the Problem Details view is specified to show, for each pattern, the user’s cumulative pattern mastery rather than the problem’s own score – keeping the two metrics visually and conceptually distinct.

2.2.5 FSRS Integration

A mapping from the 0–100 Problem Mastery score to the four FSRS review ratings has been finalised:

- 0–39 mastery → rating 1
- 40–59 mastery → rating 2
- 60–79 mastery → rating 3
- 80–100 mastery → rating 4

This rating, combined with the user’s existing FSRS review state and history, is what FSRS [YR23] uses to compute the next review date. Due items are designed to surface on the Home Dashboard’s revision flow.

2.2.6 Frontend Views Specified

The required frontend surfaces have been specified in detail:

- **Home Dashboard:** total solved problems, problems due today, patterns attempted, and a “Today’s Revision” list showing problem name, tags/patterns, cumulative pattern mastery, and a done/not-done review status.
- **Problem Library:** the full problem set with search and filters (e.g. pattern, status), keeping global problem data separate from user-specific solved status/mastery.
- **Pattern Dashboard:** a user’s cumulative mastery broken down by DSA pattern, to highlight strong and weak areas.
- **Problem Details:** problem metadata, the user’s individual problem mastery, solving statistics (time taken, submission count, runtime, memory), and per-pattern cumulative mastery shown on hover.

2.2.7 Extension Widget Specification

The floating in-page widget shown on LeetCode has been specified to display: Problem, Difficulty, tags, Time Taken, Submissions Until Success, Runtime, Memory, Efficiency Score, Your Complexity, Optimal Complexity, Space, Hints Used, a “Submit to CPRecal” action, and an “Open CPRecal Dashboard” link. A visual mock-up of this widget has been reviewed by the team as a reference for implementation.

2.2.8 Authentication and Data Isolation

CPRecal is designed as a standard authenticated application in which every user-specific record (`SolvingSession`, `Submission`, `AcceptedSolution`, `ProblemMastery`, `PatternMastery`, `FSRSState`, `ReviewHistory`) must belong to exactly one CPRecal user, while shared problem metadata remains global. It has been explicitly decided that backend-side authorization – not the frontend or the extension – must determine the authenticated user for every private read or write, so that one user can never access another user’s solving data.

Results and Evaluation

3.1 Current Status

At this stage of the project, the work completed is the requirements analysis and system design described in Chapter 2: the end-to-end data flow, the repository structure, the database schema, the Problem Mastery and Pattern Mastery formulas, the FSRS rating mapping, and the specification of every frontend view and the extension widget. These have been documented as stable references (`ARCHITECTURE.md` and `PROJECT_CONTEXT.md`) so that implementation can proceed consistently across the team and be handed off to, or picked up by, any contributor without ambiguity.

3.2 Design Verification

Since implementation and formal testing have not yet begun, verification at this stage has consisted of walking through the 17-step solving flow (Section 2.1.1) end-to-end on paper to confirm that every raw data point needed by the Problem Mastery formula, the Pattern Mastery aggregation, and the FSRS rating conversion is in fact collected somewhere upstream, and that the responsibility split between the extension, the LLM, and the backend leaves no step without an owner. This walkthrough did not surface any missing data dependency in the current design.

3.3 Pending Work

Coding of the individual modules (extension collectors, backend controllers/services, and the React frontend), along with unit, integration, and performance testing, has not yet started and will follow directly from the design finalised in Chapter 2.

Conclusion

4.1 Summary of Work

At this stage, the CPRecal project has moved from an initial idea to a fully specified system design. The three objectives set out in Section 1.2 – automated data capture, LLM-based complexity analysis feeding into a weighted mastery score, and FSRS-driven spaced revision – have each been translated into a concrete, documented design: a monorepo structure, a MongoDB schema separating shared and user-specific data, an explicit Problem Mastery formula, a clear Problem-Mastery-versus-Pattern-Mastery distinction, an FSRS rating mapping, and a full specification of every frontend view and the extension widget.

4.2 Key Findings

- **Finding 1:** Separating *problem* mastery from *pattern* mastery early in the design avoids conflating a single lucky/unlucky attempt with a user’s broader command of a DSA pattern, and this distinction has been threaded consistently through the database schema, the mastery formulas, and every frontend view.
- **Finding 2:** Keeping the Tampermonkey extension a pure collector/display layer, with the backend as the sole source of truth for mastery and review state, keeps the architecture flexible – for instance, the LLM call can be moved from the extension to the backend later (for API-key or reliability reasons) without changing how mastery or FSRS scheduling work.
- **Finding 3:** Retaining raw observations (sessions, submissions, accepted code) rather than only the derived mastery score means the scoring formula can be revised later and applied retroactively, which was an important design decision made before any implementation began.

4.3 Future Work and Recommendations

1. Implement the Tampermonkey collectors and the extension widget per the specification in Section 2.2.6.
2. Implement the backend models, controllers, and the mastery/FSRS/ingestion services per the module structure in Section 2.2.1.

3. Implement the React dashboard, problem library, pattern dashboard, and problem-details pages per Section 2.2.5 [Met24].
4. Finalise the exact normalisation formulas within each Problem Mastery component (Section 2.2.3).
5. Begin unit and integration testing once the above modules are implemented.

4.4 Final Remarks

The design phase reported here was necessary before writing implementation code, since it fixes the data contracts between the extension, the LLM, and the backend, and avoids rework once coding begins. The next report will cover the implementation undertaken against this design.

Bibliography

- [Aut24] Automattic, Inc. *Mongoose: Elegant MongoDB Object Modeling for Node.js*. 2024. URL: <https://mongoosejs.com/> (cit. on p. 8).
- [Bin24] Jan Biniok. *Tampermonkey Documentation*. 2024. URL: <https://www.tampermonkey.net/> (cit. on p. 8).
- [Met24] Meta Platforms, Inc. *React – The Library for Web and Native User Interfaces*. 2024. URL: <https://react.dev/> (cit. on pp. 8, 14).
- [Mon24] MongoDB, Inc. *MongoDB: The Developer Data Platform*. 2024. URL: <https://www.mongodb.com/> (cit. on p. 8).
- [Ope24] OpenJS Foundation. *Express – Node.js Web Application Framework*. 2024. URL: <https://expressjs.com/> (cit. on p. 8).
- [YR23] Jarrett Ye and Open Spaced Repetition. *FSRS: Free Spaced Repetition Scheduler*. 2023. URL: <https://github.com/open-spaced-repetition/fsrs4anki> (cit. on pp. 5, 8, 9).